

# CSS Interview Questions & Answers

20 questions spanning basic to advanced CSS concepts

## Basics

### 1. What is CSS, and how is it linked to an HTML document?

CSS (Cascading Style Sheets) is a language used to control the visual presentation of HTML content — layout, colors, fonts, spacing, and so on. It can be linked externally via a `<link>` tag in the `<head>`, embedded inline within a `<style>` tag, or applied directly to an element via the `'style'` attribute, with external stylesheets being the most maintainable approach.

### 2. What is the CSS box model?

The box model describes how every element is rendered as a rectangular box made of four layers, from inside out: content, padding, border, and margin. An element's total rendered size is the sum of these layers, which matters for calculating layout dimensions accurately.

### 3. What is the difference between `'box-sizing: content-box'` and `'box-sizing: border-box'`?

With `content-box` (the default), width/height apply only to the content area, so padding and border add extra size on top. With `border-box`, width/height include padding and border, meaning the box stays a predictable total size regardless of padding — which is why many developers reset all elements to `border-box` globally.

### 4. What is specificity in CSS, and how is it calculated?

Specificity determines which CSS rule wins when multiple rules target the same element. It's calculated as a weighted score based on selector types: inline styles rank highest, followed by ID selectors, then classes/attributes/pseudo-classes, then element/pseudo-element selectors. Higher specificity overrides lower specificity regardless of source order, though !important overrides normal specificity entirely.

### 5. What is the difference between relative, absolute, fixed, and sticky positioning?

'relative' positions an element relative to its own normal position, without removing it from document flow. 'absolute' removes it from flow and positions it relative to the nearest positioned ancestor (or the document if none exists). 'fixed' positions it relative to the viewport, staying in place during scroll. 'sticky' behaves like relative until a scroll threshold is crossed, then acts like fixed within its containing block.

## Layout

### 1. What is the difference between Flexbox and CSS Grid?

Flexbox is a one-dimensional layout model, designed for arranging items in a single row or column and distributing space between them. CSS Grid is a two-dimensional layout model, letting you define both rows and columns simultaneously and place items precisely within that grid. Flexbox suits component-level layouts like navbars; Grid suits overall page or complex layout structures.

### 2. How do you center a div both horizontally and vertically?

With Flexbox, set the parent to `display:flex`, `justify-content:center`, and `align-items:center`. With Grid, set the parent to `display:grid` and `place-items:center`. Alternatively, an absolutely positioned child can be

centered using `top:50%, left:50%, and transform:translate(-50%, -50%)`.

### 3. What is the difference between 'display: none' and 'visibility: hidden'?

'display: none' removes the element entirely from the document's rendering flow, so it takes up no space and other elements shift to fill the gap. 'visibility: hidden' hides the element visually but preserves its space in the layout — the element is invisible but still occupies its normal position.

### 4. What are CSS Grid's 'fr' units, and how are they used?

The 'fr' (fractional) unit represents a fraction of the remaining available space in a grid container. For example, `grid-template-columns: 1fr 2fr` divides the row into three equal parts, giving the second column twice the width of the first, after any fixed-size tracks have been accounted for.

### 5. What is a stacking context, and how does 'z-index' interact with it?

A stacking context is a conceptual layering group formed by certain properties (e.g. position with a z-index value, opacity less than 1, or transform). 'z-index' only compares elements within the same stacking context; an element with a high z-index can still be rendered below another element if it's inside a different, lower-priority stacking context created by an ancestor.

## Responsive & Selectors

### 1. What are media queries, and how are they used for responsive design?

Media queries let you apply CSS rules conditionally based on characteristics of the viewport or device, such as width, orientation, or resolution — e.g. `@media (max-width: 768px) { ... }`. They're the core mechanism behind responsive design, allowing layouts to adapt to different screen sizes.

### 2. What is the difference between 'em' and 'rem' units?

'em' is relative to the font-size of its parent element, which means nested elements can compound unexpectedly. 'rem' (root em) is always relative to the font-size of the root (`<html>`) element, giving more predictable, consistent scaling across a document regardless of nesting.

### 3. What is the difference between a pseudo-class and a pseudo-element?

A pseudo-class (single colon, e.g. `:hover`, `:first-child`) selects an element based on its state or position, without adding new content. A pseudo-element (double colon, e.g. `::before`, `::after`) targets and can generate a specific sub-part of an element, such as inserting generated content before or after the element's actual content.

### 4. What is a mobile-first approach to writing CSS?

Mobile-first means writing base styles for small screens first, then using min-width media queries to progressively add or override styles for larger viewports. This approach tends to result in leaner CSS for mobile devices and reflects the constraint that smaller-screen layouts are usually simpler to build up from.

### 5. What are CSS custom properties (variables), and what advantage do they offer over preprocessor variables?

Custom properties (e.g. `--primary-color: #333;`, used via `var(--primary-color)`) are native CSS variables. Unlike Sass/LESS variables, which are compiled away at build time, custom properties are live in the

browser — they can be read and changed dynamically via JavaScript and respect the cascade, meaning their value can differ depending on where in the DOM tree they're accessed.

## Advanced

### 1. What causes a browser to reflow versus repaint, and why does it matter for performance?

A reflow (layout) happens when a change affects an element's geometry — size, position, or the layout of other elements — and is expensive because it can cascade through the document. A repaint happens when a change affects only visual appearance (e.g. color) without affecting layout, which is cheaper. Minimizing reflows, e.g. by batching DOM changes or animating transform/opacity instead of top/left, improves rendering performance.

### 2. What is the CSS cascade, and in what order are conflicting rules resolved?

The cascade determines which of several conflicting declarations applies, resolved in this order: importance (!important rules win), then origin (author styles generally beat user-agent defaults), then specificity, and finally source order (later rules win ties).

### 3. What are CSS animations versus CSS transitions?

A transition smoothly interpolates a property's value between two states over time, typically triggered by a state change like :hover, and requires a start and end state but no intermediate keyframes. An animation, defined with @keyframes, allows multiple intermediate steps, can run automatically without a trigger, and can loop or run in a specified direction.

### 4. What is the purpose of 'will-change', and what's a risk of overusing it?

'will-change' hints to the browser that a property is about to change, letting it optimize in advance (e.g. by promoting the element to its own compositing layer). Overusing it on many elements can backfire, consuming excessive memory and GPU resources, which can actually degrade performance rather than improve it.

### 5. What is the difference between CSS Modules and CSS-in-JS, at a conceptual level?

CSS Modules are regular CSS files processed at build time so that class names are automatically scoped/hashed to be locally unique, preventing global namespace collisions while still writing plain CSS. CSS-in-JS writes styles directly within JavaScript/component files, generating scoped styles at runtime or build time and allowing styles to easily reference JavaScript variables and component props directly.